Final Presentation Prep: AI Incident Intelligence Platform

Oct 21, 2026 · @RED

1. Readiness verdict

Yes, the project is ready to present tomorrow as a live local demo. Every stage of the pipeline was built and confirmed working end to end, including a real log sent through the `/ingest` API that was flagged, matched to a past incident and explained by the LLM. I cannot see your laptop from here, so run the 5-minute check below tonight to confirm it still behaves the same.

Completion against the plan

Area	Status
Log ingestion (generator and real `/ingest` API)	Done
Rule-based and ML (Isolation Forest) detection	Done
Measured evaluation (precision, recall, F1)	Done
RAG retrieval with pgvector	Done
LLM explanation (Ollama, llama3.2)	Done
Severity triage and Slack alerts	Done
Feedback loop that updates the vector memory	Done
Docker Compose, one-command start	Done
React dashboard	Done (your redesign)
Public cloud deployment	Not done (card issue blocked it)
Real company as a pilot user	Not done

Pre-flight check (tonight, about 5 minutes)

1. Open Docker Desktop and wait until the engine says it is running.
2. Start the stack and confirm every container is Up (Postgres should say healthy):

```
cd C:\Users\Lenovo\incident-intelligence
docker compose --env-file .env up -d
docker ps
```

3. Confirm Ollama is running and has the model. Ollama runs on your laptop, outside Docker, so if it is off the dashboard will show anomalies with no explanations:

```
ollama list
```

4. Open `http://localhost:5173`, click one anomaly row, and confirm the explanation and the matched past incident both appear. I never saw the fix after your dashboard redesign, so this is the one thing to verify with your own eyes.
5. Send one test log through `/ingest` and watch it appear on the dashboard (command is in the demo script at the end).
6. Check `.env`. The `SLACK_WEBHOOK_URL` line was commented out at one point. If you want a live Slack alert in the demo, make sure it is active and uses the new, rotated webhook.

Backup plan. Record a 2-minute screen video of the working demo tonight and keep your dashboard, RAG and evaluation screenshots in one folder. If Docker or Wi-Fi misbehaves tomorrow, you show the video and keep talking.

One rule. Do not run `eval_detectors.py` live. It is only valid right after you copy a fresh `ground_truth.jsonl` and restart the services, and it can show nonsense otherwise. Quote the numbers you already measured (section 4).

2. The 30-second pitch and why we built it

The platform reads an application's logs, spots the abnormal ones, finds the most similar past incident, and has an AI write a 2 to 3 sentence explanation of what likely went wrong, then alerts the team on Slack. Engineers get an answer to "what is this and what did we do last time?" instead of a raw error.

Say it like this: "When a production system breaks, engineers drown in logs and alerts. Existing tools tell you something is wrong, but not what it means or what fixed it last time. Our platform detects the problem, remembers past incidents, and explains the new one in plain English, automatically."

Why we built it

- Every software company runs production systems, and on-call engineers lose hours on the first step of triage: understanding what an alert means.

- That knowledge usually sits in a few senior engineers' heads and leaves when they do. We store it as searchable memory that keeps growing.
- It combines the skills the course and the job market care about in one system: machine learning, RAG, system design, data pipelines and containerization.
- It is a real daily-use tool, not a toy: it accepts real logs through an authenticated API.

How it flows, in one line

Logs arrive (generator or `/ingest` API) → Redis queue → Postgres → rule detector and ML detector → vector search over past incidents → local LLM writes the explanation → severity triage and Slack alert → dashboard → engineer feedback goes back into the memory.

3. Main files and what each one does

Eleven containers run from one `docker-compose.yml`. Each backend script is its own small service, so one can fail or restart without taking down the rest.

File	Stage	What it does
`fake_log_generator.py`	Ingestion	Simulates web traffic with injected anomalies; writes `ground_truth.jsonl` (labeled anomaly windows) used for evaluation and pushes logs into Redis
`api.py` (FastAPI)	Ingestion and serving	Endpoints for stats, anomalies, recent logs, system health, feedback, and the API-key protected `/ingest` for real external logs
`redis_to_postgres_consumer.py`	Storage	Reads the Redis stream and saves each log into the Postgres `logs` table
`log_analyzer.py`	Detection (rules)	Explainable rules: 5xx status, latency above 2000 ms, crash keywords; also mines log templates with Drain3

File	Stage	What it does
`ml_anomaly_detector.py`	Detection (ML)	Isolation Forest retrained on the last 3000 logs; flags statistical outliers the rules miss
`seed_incidents.py`	RAG memory	Seeds 12 past incidents as 384-number embeddings in the `incidents` table (pgvector)
`incident_explainer.py`	RAG and LLM	Embeds a new anomaly, retrieves the closest past incident, asks Ollama (llama3.2) for a 2 to 3 sentence explanation
`triage_and_alert.py`	Alerting	Simple severity rules (critical, warning, info) and Slack webhook messages
`feedback_processor.py`	Learning loop	Takes engineer corrections and stores them as new incidents, so retrieval improves over time
`eval_detectors.py`	Evaluation	Compares each detector to the labeled ground truth and reports precision, recall and F1
`test_retrieval.py`	Demo	One-shot RAG test: shows ranked similar incidents with similarity scores
`Dockerfile`, `Dockerfile.dashboard`, `docker-compose.yml`	Deployment	Python image for all services, React build served by nginx, and the orchestration with health checks and auto-restart
`App.jsx`, `IncidentTable.jsx`	Dashboard	Live-polling React UI: stat cards, anomaly chart, expandable rows with the AI explanation, thumbs up/down feedback

Database in one breath: `logs` (raw records), `anomalies` (what was flagged, by which detector, the explanation, the feedback), `incidents` (the vector memory), plus a Redis stream as the queue between ingestion and storage.

4. Pros

Measured results you can quote (from your own runs):

Measure	Value
Evaluation set	1,141 logs, 20 labeled anomaly windows, 113 true anomalous logs
Rule detector (`latency_spike`)	Precision 1.00, recall 0.47, F1 0.64
Rule detector (`error_status_code`)	Precision 1.00, recall 0.53, F1 0.69
ML detector (Isolation Forest)	Precision 0.40, recall 0.99, F1 0.57
RAG retrieval	Correct incident ranked first at 72.7% similarity, unrelated one lowest at 48.5%
ML scoring speed	1,000 rows in 0.0089 s, about 112,000 rows per second
Run totals at last count	1,873 logs, 650 anomalies (461 ML, 103 error status, 92 latency)

The honest reading: rules almost never raise a false alarm but miss about half the real problems; the ML catches nearly everything but also flags normal logs. Together they cover each other's weakness.

Strengths

- **End to end, not a notebook.** Ingestion, detection, memory, explanation, alerting, feedback and dashboard all run together.
- **Two detectors, evaluated honestly.** Real precision and recall against labeled data, with the tradeoff reported instead of hidden.
- **Explains, not just alerts.** The LLM answer is grounded in a retrieved past incident, which reduces made-up answers.
- **Self-improving.** Engineer corrections become new memory entries; confirmed working ("Learned from anomaly #2309").
- **Private and free.** The LLM runs locally through Ollama, so logs never leave the machine, and every tool is free or open source (cost: $0).
- **Runs on real data.** The authenticated `/ingest` endpoint accepted an outside log that flowed through the whole pipeline.
- **Reproducible.** One `docker compose up` starts all 11 containers, with health checks and automatic restarts.

- **Modular.** Each stage is a separate service, so any part can be swapped (a different model, a different detector) without rewriting the rest.
- **Explainable triage.** Severity uses plain rules on purpose, so an engineer can see exactly why something was marked critical.

5. Cons and limitations (say these before they are asked)

Examiners respect a student who names the weak points first. Each limit below comes with what you would do about it.

Limitation	Why it exists	What you would do next
Trained and tested mostly on synthetic logs	No real company data yet; the generator injects the anomalies, so ground truth is known	Run a pilot on a real service's logs and re-evaluate
ML precision is only 0.40	Isolation Forest is unsupervised and flags anything statistically unusual, not only real faults	Tune `contamination`, add features, use feedback labels to train a supervised second stage
Rules miss about half the real anomalies	Fixed thresholds cannot see subtle patterns	Keep rules for precision, let ML cover recall
Only 3 simple features (latency, status code, missing latency)	Kept small to stay explainable and fast	Add message text embeddings, request rate, per-route baselines
Classic RAG only	One embedding per incident, top-k retrieval, single LLM pass; no reranking or hybrid search	Add reranking, hybrid keyword plus vector search, multi-incident synthesis
Small knowledge base (12 seeded incidents plus feedback)	Seed data was written by hand	Import real postmortems and ticket history
Small local LLM (llama3.2)	Chosen to be free and private	Allow a larger or hosted model when privacy allows; add an answer-quality check
LLM output is not formally evaluated	We measured detection and retrieval, not explanation quality	Rate explanations with engineer feedback and a small labeled set

Limitation	Why it exists	What you would do next
Runs on one machine, not in the cloud	Cloud deployment was blocked by a card issue	Deploy on a free cloud VM; move to Kubernetes for scale
Single-node Postgres and Redis	Fine for a demo, not high availability	Replicas, managed services, a real message broker such as Kafka
No user accounts or multi-tenancy	Out of scope for the prototype	Add login, per-team data separation
Only the `/ingest` endpoint reads real logs	No agents or connectors for real log tools yet	Build forwarders for common sources (Fluent Bit, CloudWatch, Datadog export)
Depends on a laptop-hosted Ollama	The LLM sits outside Docker	Put the model in its own container or service
Slack alerts are one-way	No acknowledge or resolve flow	Add interactive buttons and paging integration

6. Where it stands against other tools

Our project is a prototype of the explanation-and-memory layer, not a replacement for a full monitoring suite. Position it as the piece that sits on top of tools like these. The descriptions are general characterizations from my own knowledge, not freshly checked against vendor pages, so keep your claims modest.

Tool	What it is good at	What our approach adds or does differently
Datadog, Dynatrace	Full observability suites with built-in anomaly detection and AI assistants	We are open, free and private; we show the full mechanism (detector, retrieval, LLM) and learn from our own team's incident history. They are far larger and more mature.
Splunk, Elastic (ELK)	Powerful log search, dashboards and some ML anomaly detection	They find and display logs; we add "similar past incident plus plain-English explanation" on top of detection

Tool	What it is good at	What our approach adds or does differently
Grafana, Prometheus	Metrics dashboards and threshold alert rules	Thresholds alone cannot say what an alert means; ours attaches an explanation and a precedent
PagerDuty, Opsgenie	Paging, on-call schedules, escalation	They get the right person woken up; we help that person understand the problem faster. The two are complementary.
Sentry	Application error tracking with stack traces	Covers code exceptions; ours covers system behavior (latency, status patterns) and uses past incidents
Research prototypes (RAGLog, LogRAG style papers)	Show RAG helps log analysis	We built a working, containerized, end-to-end system with feedback and live ingestion, plus a measured rule-versus-ML comparison

One-line positioning: "Existing tools tell you that something is wrong. We tell you what it probably is and what fixed it last time."

Where we are weaker (admit this): scale, number of integrations, UI polish, and years of production hardening. We win on transparency, privacy, cost and the learn-from-your-own-history loop.

7. What makes it different, industry use, and the future

What makes it different

1. **It remembers.** Past incidents are stored as vectors, so the system matches by meaning, not keywords (a "checkout page failing" log matched "payment connection pool exhaustion" at 72.7%).
2. **It explains.** The output is a short plain-English cause grounded in a real precedent.
3. **It learns from the team.** A thumbs-down with a correction becomes new memory, with no retraining of any model.
4. **It uses two detectors on purpose** and publishes the tradeoff between them.

5. **It is private by design.** Local LLM, local database, no log data sent to a third party.
6. **It is real.** Authenticated ingestion, one-command deployment, health checks.

How an industry team would use it

- **On-call triage:** an alert arrives on Slack already carrying severity, a likely cause and a link to the similar past incident.
- **Onboarding:** new engineers get the team's incident history as searchable, explained memory.
- **Postmortems:** the feedback loop turns each resolved incident into a reusable entry.
- **Integration path:** forward logs from an existing system to `/ingest` (a webhook, a log shipper, or a small script), then read results on the dashboard or in Slack. It sits beside the existing monitoring stack instead of replacing it.
- **Good fit:** small and mid-size engineering teams, startups, and teams that cannot send logs to outside vendors for privacy or cost reasons.

Future roadmap

Horizon	Work
Next (weeks)	Deploy on a free cloud VM; run a pilot on one real service's logs; re-run the evaluation on real data
Short term	Reranking and hybrid search; more features for the ML detector; explanation-quality scoring from feedback
Medium term	Connectors for common log sources; multi-user accounts and per-team memory; interactive Slack actions and paging integration
Long term	Supervised second-stage model trained on feedback labels; automatic root-cause suggestions across services; a hosted product for small teams

8. Cross-questions and answers

Keep answers to 3 or 4 sentences. If you do not know, say what you would test to find out.

The problem and the idea

Q: What problem does this solve, in simple words? A: When a system breaks, engineers see a flood of logs and alerts and have to work out what it means. We automatically detect the abnormal log, find the most similar past incident, and explain it in plain English.

Q: How is this different from Datadog or Splunk? A: Those are large platforms that collect, search and alert. We are the explain-and-remember layer: similar past incident plus a short explanation, learning from the team's own corrections, running privately and free. We do not claim to replace them.

Q: Is this just a wrapper around an LLM? A: No. The LLM is the last step. Ingestion, queueing, two detectors, vector retrieval, triage, the feedback loop and the evaluation are the system. The LLM only turns retrieved facts into a readable sentence.

Q: Who would pay for or use this? A: Small and mid-size engineering teams with on-call duty, especially ones that cannot send logs to outside vendors. Pilot first, then a hosted or self-hosted offering.

Machine learning and evaluation

Q: Why Isolation Forest? A: It is unsupervised, so it needs no labeled anomalies, which real companies rarely have. It is fast, runs on CPU, and isolates outliers by how few random splits they need.

Q: Why not deep learning such as DeepLog or an LSTM? A: Those need much more data and tuning and are harder to explain. For a first system with little real data, a simple, fast, explainable model is the right start. It is on the roadmap once feedback gives us labels.

Q: Your ML precision is only 0.40. Is that bad? A: It is a tradeoff, not a failure. The ML catches 99% of real anomalies but flags some normal logs too. The rules are 100% precise but find only about half. In incident response a missed outage costs more than an extra check, so we keep both and report the tradeoff.

Q: Explain precision, recall and F1. A: Precision: of what we flagged, how much was truly bad. Recall: of everything truly bad, how much we caught. F1 is the balance of the two.

Q: How did you get ground truth? A: The generator injects anomalies in known time windows and writes them to `ground_truth.jsonl`. A log counts as truly anomalous if its timestamp falls inside a window.

Q: Is the evaluation realistic? The data is synthetic. A: Fair point, and a real limitation. Injected faults are easier to spot than real ones, so recall on real data will likely be lower. We have shown the method and the pipeline; the next step is a pilot on real logs.

Q: What does the `contamination` setting do? A: It tells Isolation Forest what fraction of data to treat as anomalous. We lowered it from 0.15 to 0.05 because 0.15 flagged far too much.

RAG and the LLM

Q: What type of RAG is this? A: Classic RAG: one embedding per document, top-k cosine similarity search, one LLM call. Not advanced RAG (no query expansion, reranking or hybrid search). We say so openly and list those as upgrades.

Q: What are embeddings, and why this model? A: An embedding turns text into a list of numbers so similar meanings are close together. We use `all-MiniLM-L6-v2` (384 numbers) because it is small, free, fast on a CPU and good enough for short log text.

Q: Why pgvector instead of Pinecone or a dedicated vector database? A: We already needed Postgres, so pgvector gives vector search with no extra service, free, and queryable alongside our other tables. At our scale it is more than enough.

Q: How do you stop the LLM from making things up? A: We give it the retrieved incident as context and force a short answer (2 to 3 sentences, no extras). That reduces but does not remove errors, so the explanation is a hypothesis for an engineer to confirm, and the dashboard shows the matched incident next to it.

Q: What if no similar incident exists? A: Then the match will have a low similarity score and the explanation is less reliable. A minimum-similarity cutoff that says "no close precedent found" is the right fix and a good next step. Check how `incident_explainer.py` handles this before the presentation so you can answer exactly.

Q: Why a local LLM and not ChatGPT or Claude via API? A: Privacy and cost. Production logs can contain sensitive data, and a local model keeps it on the machine and costs nothing. The code can switch to a stronger hosted model where policy allows.

Q: Is the explanation always correct? A: No, and we do not claim it is. That is why there is a thumbs up/down control, and corrections are stored for future matches.

Architecture and engineering

Q: Why Redis between ingestion and the database? A: It decouples the two. If the database is slow or restarting, logs wait in the stream instead of being lost, and producers never block on database writes.

Q: Why not Kafka? A: Kafka is the right choice at large scale but heavy to run. Redis Streams give the same pattern with far less setup. Moving to Kafka is a straightforward upgrade.

Q: Will it scale? A: Detection scoring is very fast (about 112,000 rows per second in our test). The real bottlenecks are single-node Postgres and the LLM, which takes seconds per explanation. We process newest anomalies first and would add replicas, batching and a larger inference setup to scale.

Q: Why Docker Compose? A: So the whole 11-service system starts with one command and behaves the same on any machine, with health checks and automatic restarts.

Q: What was the hardest bug? A: The dashboard hung on startup. The cause was several services holding open database transactions that blocked the API's table update. We found it by adding a lock timeout that turned a silent hang into a clear error, then set autocommit on those connections.

Security, data and ethics

Q: How is the `/ingest` endpoint secured? A: It requires an API key in a request header and returns 401 without it. Be ready to say it is basic: the next steps are HTTPS, per-client keys and rate limiting.

Q: What about secrets? A: They live in a `.env` file that is git-ignored. We once committed a Slack webhook by mistake, GitHub's scanning blocked it, and we rotated it. Since then secrets stay out of every tracked file.

Q: Could the feedback loop be abused or poisoned? A: Yes, a wrong correction becomes a wrong memory. With real users we would add review or approval of corrections and per-user trust.

Q: What about privacy of the logs? A: Everything, including the LLM, runs locally, so no log data leaves the machine.

Honest-limits and next steps

Q: Have you tested it on real company data? A: Not at scale. We sent a real external log through `/ingest` and it flowed through the whole pipeline, but the evaluation uses synthetic data. A real pilot is the next step.

Q: What would you do with two more months? A: Cloud deployment, a pilot on a real service, reranking and hybrid retrieval, a better feature set for the ML, and explanation-quality scoring from feedback.

Q: Was this a team project? What did you personally do? A: I built it solo end to end: architecture, pipeline, models, RAG, Docker, dashboard and evaluation. Be ready to explain any file, because the examiner may ask you to open one.

Q: What did you learn? A: That a working system is mostly about reliability and honest measurement: queues, health checks, stale-data bugs in evaluation, and reporting a tradeoff truthfully rather than a clean but misleading number.

9. Five-minute demo script

Minute	Do this	Say this
0:00	Slide: the problem	"Engineers drown in logs. Tools say something broke, not what it means."
0:45	Slide: architecture or the flow line from section 2	"Logs go through a queue, two detectors, a memory of past incidents, and an LLM that explains."
1:30	Open the dashboard at `http://localhost:5173`	Point at the stat cards, the anomaly chart and the pipeline health strip
2:15	Send the live test log (command below), then refresh	"This is a log I just sent from outside. It gets flagged, matched to a past incident and explained."
3:00	Click that row to expand it	Read the explanation and the matched incident aloud, point at the similarity score
3:45	Click thumbs down and add a correction (optional)	"Corrections become new memory, so it improves."
4:15	Show the evaluation slide or screenshot	"Rules: precise but narrow. ML: catches almost everything but noisier. We keep both."
4:45	Close	State limits and next step: real pilot and cloud deployment

Live test log (PowerShell, use the key that is in your `.env`):

```
Invoke-RestMethod -Uri "http://localhost:8000/ingest" -Method Post -Headers @{"X-API-Key"="<your INGEST_API_KEY>"} -ContentType "application/json" -Body '{"route": "/checkout", "status_code": 500, "response_time_ms": 3200, "message": "payment gateway timeout"}'
```

If it returns `status: accepted`, wait a few seconds and refresh. If it returns 401, the key does not match `.env`; check it with `docker exec incident-intelligence-api-1 printenv INGEST_API_KEY` (use the container name shown by `docker ps`).

Optional RAG proof if time allows: `python test_retrieval.py` shows the ranked matches with similarity scores.

Do not run `eval_detectors.py` live (see section 1). **Do** keep the screen-recording backup open in another window.